



FleetCore Fleet Management Hackathon

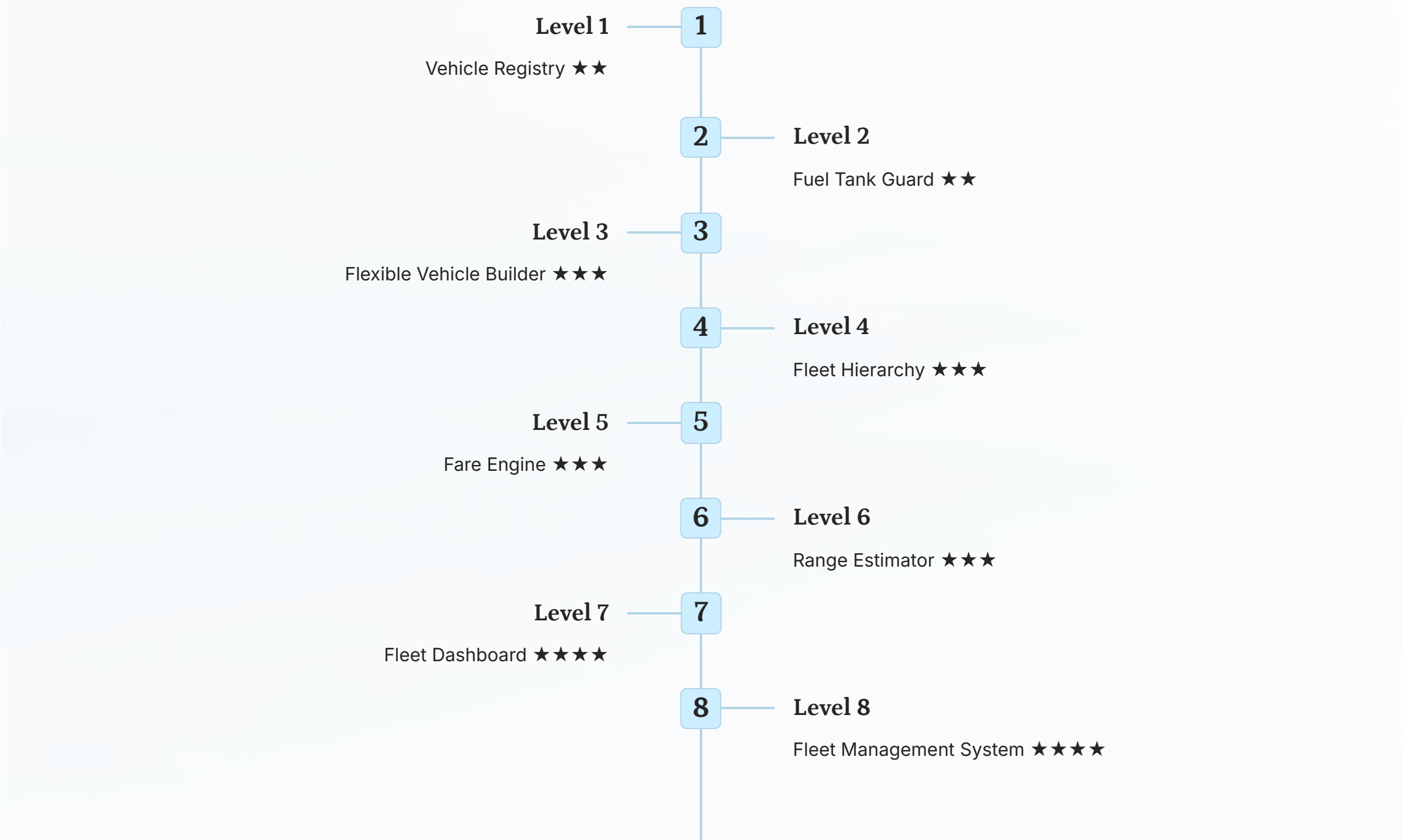
Object-Oriented Programming · Problem Statements

DEVELOPED BY TALENCIAGLOBAL

About This Hackathon

FleetCore is the software behind a vehicle-fleet platform. Across eight levels you will build it up object by object: a vehicle registry, a self-guarding fuel tank, a flexible builder, a car/truck hierarchy, a fare engine, a range estimator, a fleet dashboard, and finally a complete fleet-management system.

Each level states the inputs, the required outputs and the formulas you may rely on. The levels run from intermediate to advanced and walk through the pillars of object-oriented programming — classes and objects, encapsulation, constructors, inheritance, polymorphism and abstraction — before a capstone that combines them. No starter code is provided; design and write each solution yourself.



Level 1 — Vehicle Registry

☆ ★ INTERMEDIATE

CLASSES & OBJECTS

Scenario

FleetCore registers every vehicle that joins the fleet. Model a single vehicle as an object that holds its own data — an id, a brand and a top speed — and knows how to describe itself and classify its speed.

Concepts Covered

- Defining a class with fields
- Creating an object (instance)
- Instance methods that act on the object's own data
- Returning a derived classification

Problem Statement

Given: A vehicle id, a brand and a top speed (km/h)

Required output: A one-line description, its top speed, and its speed category

Constraints: Top speed is a non-negative integer. Category is **Fast** when speed ≥ 100 , otherwise **Standard**.

Formula Guidance

```
describe -> "Vehicle {id} ({brand})"  
category: speed >= 100 -> Fast | else Standard
```

Sample Run

Input	Output
V001 Toyota 120	Vehicle V001 (Toyota)
	Top Speed: 120 km/h
	Category: Fast

⚠ The category test uses a $\geq$ threshold at 100, so a top speed of exactly 100 counts as Fast, not Standard.

ℹ A class bundles related data (id, brand, speed) with the behaviour that acts on it (describe, category), so each object knows how to present itself.

Level 2 — Fuel Tank Guard

☆ ★★ INTERMEDIATE

ENCAPSULATION

Scenario

A vehicle's fuel tank must never hold less than 0 or more than 100 litres. Wrap the fuel level in an object that guards itself: every refuel or consume request is validated before it is allowed to change the stored level.

Concepts Covered

- Private fields hidden from outside code
- Validating methods (guarded setters)
- Rejecting invalid state changes
- Read-only access through getters

Problem Statement

Given: An initial fuel level, then a number of operations — each a REFUEL or CONSUME with an amount.

Required output: The final fuel level and the number of operations that were rejected.

Constraints: Fuel level must always stay within 0..100 inclusive. An operation that would move the level outside that range is rejected and leaves the fuel unchanged.

Formula Guidance

```
refuel valid when 0 <= fuel + amount <= 100
consume valid when 0 <= fuel - amount <= 100
```

Sample Run

Input	Output
50 (initial)	Final Fuel: 40
REFUEL 30	Rejected Operations: 2
CONSUME 90	
REFUEL 25	
CONSUME 40	

⚠️ Reject the operation and leave the fuel unchanged when it would breach a bound — do not clamp the value to the nearest limit.

ℹ️ Encapsulation hides the raw field and forces every change through a validating method, so the object can never reach an invalid state.

Level 3 — Flexible Vehicle Builder

☆ ★ ★ INTERMEDIATE

CONSTRUCTORS

Scenario

FleetCore ingests vehicle records from two sources. Some records carry a full specification (brand and top speed); others carry only a brand and rely on a sensible default. Build the same class either way, and report how often the default path was used.

Concepts Covered

- Constructor overloading
- Default values for missing fields
- Constructor delegation (one constructor calling another)
- Counting which path built each object

Problem Statement

Given: A count n , then n records: FULL with a brand and speed, or DEFAULT with only a brand.

Required output: For each vehicle, its brand and resulting speed; then the number of vehicles built through the default path.

Constraints: A DEFAULT record creates a vehicle with the default top speed of **80**. A FULL record uses the speed supplied.

Formula Guidance

default speed = 80 when no speed is supplied

Sample Run

Input	Output
FULL Toyota 120	Toyota: 120
DEFAULT Honda	Honda: 80
FULL Ford 140	Ford: 140
	Default Constructors Used: 1

⚠ Route DEFAULT records through the brand-only constructor (which fills in 80) rather than hard-coding 80 at the call site, so the default is defined once.

ℹ Constructor overloading lets one class be built several ways while guaranteeing every object ends up fully initialised.

Level 4 — Fleet Hierarchy

☆ ★★★ INTERMEDIATE+

INHERITANCE

Cars and trucks share the basics — a brand and a top speed — but each adds something of its own: cars carry passengers, trucks carry cargo. Model this with a base Vehicle and two subclasses that reuse the shared parts and extend them.

Vehicle (Base Class)
Shared fields: brand, speed

Vehicle {brand}: {speed} km/h

Car (Subclass)
Extends Vehicle, adds seat count

Car {brand}: {speed} km/h, {seats} seats

Truck (Subclass)
Extends Vehicle, adds cargo capacity (kg)

Truck {brand}: {speed} km/h, {cargo} kg cargo

Concepts Covered

- A base class with shared fields
- Subclasses that extend the base
- Calling the base constructor (super)
- Overriding a method per subclass

Sample Run

Input	Output
CAR Toyota 120 5	Car Toyota: 120 km/h, 5 seats
TRUCK Volvo 90 8000	Truck Volvo: 90 km/h, 8000 kg cargo

 Call the base constructor (super) for the shared brand and speed instead of re-assigning those fields inside each subclass.

 Inheritance puts shared state in a base class and lets each subclass extend it, so common fields are written once and specialised behaviour is added on top.

Level 5 — Fare Engine

☆☆☆ ADVANCED

POLYMORPHISM

FleetCore quotes a trip fare for any vehicle in the fleet. Each vehicle type prices a trip differently, but the booking code should ask every vehicle for its fare the same way and let the object decide how to compute it.

		
Car Fare = 10 × distance Standard per-km rate for passenger cars.	Bike Fare = 5 × distance Economy rate for two-wheelers.	Truck Fare = 15 × distance + 100 Premium rate plus a flat loading surcharge.

Concepts Covered

- A base method overridden per subclass
- Dynamic dispatch (the object picks the method)
- A factory that builds the right subclass
- Treating different types through one interface

Sample Run (distance = 10)

Input	Output
CAR Toyota	Toyota (Car) Fare: 100
BIKE Hero	Hero (Bike) Fare: 50
TRUCK Volvo	Volvo (Truck) Fare: 250

⚠ Drive the fare from the object's actual type via method overriding — do not switch on the type tag inside one large function.

ℹ Polymorphism lets one fare(distance) call dispatch to the right subclass at run time, so the loop never needs to ask what type each vehicle is.

Level 6 — Range Estimator

☆ ★★★ ADVANCED

ABSTRACTION

Every FleetCore vehicle must report how far it can travel on a full charge or tank, but electric and petrol vehicles compute it from completely different inputs. Define an abstract contract that all vehicles obey while leaving the formula to each concrete type.

Concepts Covered

- An abstract base class
- Abstract methods with no body in the base
- Concrete subclasses that must implement the contract
- A base type that cannot be instantiated directly

Formula Guidance

Electric `maxRange` = battery * efficiency
Petrol `maxRange` = fuel * mileage

Language Notes

- Python: `abc.ABC + @abstractmethod`
- C++: `pure virtual method (= 0)`
- Java: `abstract class + abstract methods`

Problem Statement

Given: A count `n`, then `n` records: ELECTRIC with brand, battery and efficiency, or PETROL with brand, fuel and mileage.

Required output: For each vehicle, a line of the form `brand (kind)`
Range: value km.

Sample Run

Input	Output
ELECTRIC Tesla 75 6	Tesla (Electric) Range: 450 km
PETROL Honda 40 15	Honda (Petrol) Range: 600 km

⚠ The base class only declares `maxRange()`; it has no formula of its own and must not be instantiated directly.

ℹ Abstraction defines a contract — every vehicle has a maximum range — while leaving each subclass free to compute it its own way.

Level 7 — Fleet Dashboard

☆☆☆☆ ADVANCED

POLYMORPHISM + AGGREGATION

FleetCore's dispatch dashboard summarises a mixed fleet for a planned trip: the total fare it would bill, the single highest-earning vehicle, and how the fleet breaks down by type. The summary loop should treat every vehicle uniformly.



Total Fare

Sum of every vehicle's fare(distance) across the entire fleet.



Highest Fare

The vehicle with the greatest individual fare. First vehicle wins on a tie.



Fleet Breakdown

Count of Cars, Bikes and Trucks in the fleet.

Concepts Covered

- A collection of mixed subclass objects
- Aggregating via a common interface
- Tracking a maximum across objects
- Counting per concrete type

Fare Rules (same as Level 5)

- Car: 10 per km
- Bike: 5 per km
- Truck: 15 per km + 100 flat

Sample Run (distance = 20, 4 vehicles)

Input	Output
CAR A	Total Fare: 900
BIKE B	Highest Fare: C (400)
TRUCK C	Cars: 2
CAR D	Bikes: 1
	Trucks: 1

⚠ Keep the first vehicle on a fare tie by updating the best only when a fare is strictly greater than the current best.

ℹ Once vehicles share a common interface, a single loop can aggregate a mixed fleet without caring about concrete types.

Level 8 — Fleet Management System

✳️ ★★★★★ ADVANCED

ALL OOP PILLARS — CAPSTONE

Tie everything together into FleetCore's cost dashboard. Each vehicle keeps its own encapsulated data, is built through a constructor hierarchy, obeys an abstract cost contract, and is aggregated polymorphically into a single operating-cost report.



Encapsulation

State behind getters



Inheritance

Constructor hierarchy (super)



Abstraction

Abstract cost/category contract



Polymorphism

Polymorphic aggregation over the fleet

Cost Formulas

Car cost = dailyKm * 8

Bike cost = dailyKm * 3

Truck cost = dailyKm * 12 + 200

Categories

- Car → Passenger
- Bike → Passenger
- Truck → Freight

Average Formula

average = total / n (2 decimals)

Sample Run (4 vehicles)

Input	Output
CAR Toyota 100	Total Operating Cost: 4750
BIKE Hero 50	Most Expensive: Volvo (2600)
TRUCK Volvo 200	Passenger Vehicles: 3
CAR Ford 150	Freight Vehicles: 1
	Average Cost: 1187.50

 Compute the average from the integer total divided by n and print it to exactly two decimals (1187.50, not 1187.5).

 The capstone combines every pillar — encapsulated state, a constructor hierarchy, an abstract contract, and polymorphic aggregation — into one small system.